

Советы начинающему программисту

Сборник советов, которые я хотел бы получить, когда только начинал изучать программирование и писать свои первые программы. Содержит в себе как точечные советы по оптимизации, оформлению и правильному написанию кода, так и общие советы.

Написано в свободной форме, как обращение самому себе.

1. Всегда пиши комментарии. Поверь, мне это сэкономит тебе кучу времени в будущем, когда ты снова полезешь в свой код, чтобы что-нибудь дописать или проверить. А еще больше тебе будет благодарен человек, которому придется работать с твоим кодом после тебя.
2. List comprehension - сила. Когда есть выбор: создать простой список с помощью цикла или list comprehension, всегда выбирай второй вариант. Работает быстрее, читабельность повышается, а занимает одну строчку, вместо целого блока кода.
3. Учи встроенные функции Python. Тот же sum работает быстрее, потому что реализован с помощью C-языка и отлажен, аналогично занимает меньше места. И так со всеми встроенными функциями Python. Если есть питоничный аналог, то используй его вместо того, чтобы изобретать свой велосипед. Быстрее точно не получится, а код становится красивее и лаконичнее.
Не знаешь, где учить? Беги в документацию и прошерсти ее хотя бы на базе визуального представления какие встроенные функции и модули существуют. В идеале их все надо знать наизусть и уметь применять. Много? - ДА! Но зная их ты молниеносно будешь писать свои программы и думать только о своих алгоритмах, а не искать как эти алгоритмы реализовать.
4. Виртуальное окружение под каждый проект - это мастхев.

5. Использование git и github или аналогов - это мастхев. Также нельзя забывать про файл .gitignore, что не загрузить чувствительные данные (логины и пароли) во внешний репозиторий. Где хранить такие данные? Переменные окружения или файл .env.
6. Помни, что написать свой собственный проект - намного лучше, чем прочитать пару новых книг по программированию. Опыт будет всегда важнее знаний. Постановка собственной задачи и ее решение - намного лучше, чем задачи с leetcode.
7. Будь перфекционистом. Выделяй время на анализ своего кода и его рефакторинг.
8. Думай об архитектуре приложения заранее. Помни простое утверждение: разделяй и властвуй. Лучше написать много маленьких независимых классов, которые собираются в одно большое приложение с функционалом, чем написать весь алгоритм в 2000 строк на одном листе. После ты сможешь легко использовать наработки в других проектах, что сократит разработку кода в разы.
9. Еще про архитектуру: нарисуй ее блок схемами перед началом работы. На листочке или в Figma. Всегда держи в голове про независимость отдельных компонентов. Всегда держи в голове, что приложение может быть расширено и заранее продумай ситуацию, как ты будешь его масштабировать
10. Пишешь приложение для себя? - забей на обработку ошибок со стороны пользователя. Пользователь ты один! Никаких ошибок не будет, потому что ты знаешь алгоритм изнутри. Столько времени сэкономишь
11. Пишешь приложение для пользователей? Удели x2 времени на обработки ошибок со стороны пользователей. Продумай все варианты неправильного поведения пользователя и будь готов.

Советы по оптимизации и эффективности кода:

12. Изучи основы алгоритмов и структур данных. Это каркас любой программы. Не обязательно становиться гуру, но базовые вещи, такие как хэш-таблицы, множества и алгоритмы сортировки, знать просто необходимо.

13. Всегда тестируй производительность кода на реальных данных. Если твой алгоритм работает за секунду на 100 элементах, это не значит, что он будет работать за 10 секунд на 1000 элементах.
 14. Используй профайлеры и дебаггеры. Они спасут тебя от сотен бессонных ночей и упростят поиск узких мест твоего кода.
-

Советы по работе с кодом и IDE:

15. Подружись с IDE или редактором кода (PyCharm, VSCode). Узнай горячие клавиши и плагины, экономящие время.
 16. Используй автоматическое форматирование кода (например, black для Python). Ты удивишься, как много времени и нервов это сэкономит.
 17. Настрой линтеры (flake8, pylint), которые сразу укажут на слабые места твоего кода. Привыкай соблюдать чистоту кода с самого начала.
-

Советы по эффективному обучению и саморазвитию:

18. Учись правильно гуглить. Искусство программиста — быстро находить нужную информацию и эффективно её использовать.
 19. Веди конспекты своих открытий и найденных решений. Однажды ты очень сильно себя поблагодаришь.
 20. Пиши статьи или делись опытом публично. Когда объясняешь другим, глубже разбираешься в теме сам.
-

Советы по управлению временем и задачами:

21. Начинай каждый день с короткого списка задач на сегодня. Вычеркивай по одной и не отвлекайся на другое, пока не закончил текущую.
 22. Не стесняйся использовать метод Pomodoro (25 минут работаешь, 5 отдыхаешь). Ты не поверишь, насколько это повысит твою продуктивность.
-

Советы о карьерном росте и командной работе:

- 23. Не бойся задавать вопросы более опытным коллегам. Лучше спросить сейчас, чем терять часы на самостоятельный поиск ответа.
 - 24. Научись грамотно общаться в чатах, тикетах и код-ревью. С первых дней карьеры помни, что программирование — это командная работа, где важно уметь донести свою мысль и понять чужую.
 - 25. Всегда пиши понятные названия переменных и функций. Чем лучше названия, тем меньше комментариев придется писать.
-

Советы о практическом подходе к проектам:

- 26. Создай привычку регулярно делать резервные копии своих проектов. Никогда не знаешь, когда это спасёт тебя от потери недель работы. Как правило заведи себе использование git для каждого проекта. И регулярно делай коммиты. В таком случае даже если твой комп выйдет из строя, твои наработки останутся во внешней репозитории.
 - 27. Изучай и используй Docker. Это упростит запуск проектов и развертывание на сервере, избавит от бесконечных проблем совместимости.
 - 28. С самого начала учись работать с базами данных (PostgreSQL, MySQL, SQLite). Большинство реальных проектов связано с хранением и обработкой данных.
-

Советы для личного комфорта и предотвращения выгорания:

- 29. Позаботься о своем рабочем месте. Хороший монитор, удобный стул и качественная клавиатура сохранят твоё здоровье и желание кодить.
 - 30. Делай перерывы и не забывай про физическую активность. Лучшие идеи приходят, когда ты отдыхаешь или занимаешься спортом.
-

Советы о мотивации и философии программирования:

- 31. Никогда не сравнивай себя с другими программистами. Всегда сравнивай себя с собой вчерашним.

- 32. Прими то, что ошибки — это часть обучения. Чем больше ты ошибаешься, тем быстрее растешь. То, что сегодня для тебя кажется трудным - завтра окажется базовым действием.
- 33. Будь готов переписать свой код заново, даже если он уже «работает». Поверь, иногда легче стереть и написать заново, чем бесконечно латать старый код.

Советы о работе с инструментами и окружением:

- 34. Научись эффективно использовать терминал. Базовые команды (`ls`, `cd`, `grep`, `find`, `ssh`, `vim`, `nano`) существенно ускорят работу с проектами и серверами.
- 35. Используй систему контроля версий грамотно. Не складывай весь код в одну ветку. Делай отдельную ветку для каждой новой задачи или фичи.
- 36. Поднимай основы Continuous Integration и Continuous Deployment (CI/CD). Это даст тебе преимущество, когда твой проект вырастет. Но нам самом деле даже если твой проект еще маленький, но уже где-то в сети, то CI/CD поможет тебе не терять много времени на постоянном деплое.

Советы по работе с ошибками и исключениями:

- 37. Научись читать и интерпретировать сообщения об ошибках (tracebacks). Чем быстрее научишься, тем быстрее найдёшь и исправишь проблему. Отметай базовый набор действий “Копировать-Вставить в гугл-Смотреть решение”. Сначала попробуй сам вчитаться и понять суть ошибки. В большинстве случаев, там ничего трудного.
- 38. Пиши понятные сообщения об ошибках в своих программах. Это сильно упростит отладку в будущем.
- 39. Не бойся пользоваться StackOverflow, но не копируй слепо код. Всегда разбирайся, как он работает, и адаптируй под свои нужды.

Советы по работе с внешними библиотеками и документацией:

40. Выбирая внешнюю библиотеку, оценивай её популярность, поддержку и активность сообщества. Это позволит избежать ситуации, когда библиотека станет заброшенной.
 41. Научись читать исходный код популярных библиотек. Это отличный способ быстро прокачаться в программировании и понять «кухню» других разработчиков.
 42. Не бойся читать официальную документацию. Там часто написано чётче, понятнее и быстрее, чем в любых статьях и уроках.
-

Советы по безопасности:

43. Никогда не храни пароли, API-ключи и секретные данные в коде. Используй переменные окружения или специальные файлы, которые не коммитятся. (Или еще раз перечитай п.5)
 44. Всегда проверяй входные данные от пользователей. Даже если думаешь, что ты сделал это уже в другом месте. Причем входные данные лучше проверить сразу при их "входе". В одном месте. И если они сразу невалидные, отдать пользователю сообщение об ошибке.
-

Советы про ментальное здоровье и баланс жизни:

45. Учись говорить «нет». Не берись за все задачи сразу. Лучше сделать меньше, но качественно, чем много и поверхностно.
 46. Имей увлечения и интересы помимо программирования. Так ты будешь меньше выгорать и сохранять креативность.
 47. Не бойся брать отпуск или перерыв от программирования. Важно перезагружаться, чтобы твоя продуктивность и мотивация оставались высокими.
-

Советы про карьеру:

48. Создавай и поддерживай своё портфолио. Это твоя визитная карточка при устройстве на работу или фриланс.

- 49. Не переставай учиться новому, даже если кажется, что уже знаешь достаточно. IT – это постоянное развитие и обновление навыков.
- 50. Найди ментора или наставника. Даже один мудрый совет может сэкономить тебе годы экспериментов и поисков. Плюс если задача очень сложная и ты долго не можешь ее решить, то ментор сможет подсказать тебе верное направление.

Советы по качеству кода и его поддержке:

- 51. Научись писать автоматические тесты. Unit-тесты помогут тебе быть уверенным, что новая правка не сломала старый функционал.
- 52. Запомни принцип DRY («Don't Repeat Yourself»). Если один и тот же код повторяется больше двух раз, сделай функцию или отдельный модуль. Еще есть крутой но чуть более сложный принцип SOLID.

Советы по командной разработке:

- 53. Учись делать грамотные код-ревью. Это прокачает тебя не меньше, чем написание собственного кода.
- 54. Пиши документацию к своему проекту сразу, а не когда он разрастётся и ты забудешь половину деталей. (Перечитай п.1)

Советы про производительность и ресурсы:

- 55. Следи за потреблением ресурсов своей программой (CPU, память, дисковое пространство). Умение видеть узкие места важно даже в маленьких проектах.

Советы по выбору технологий и фреймворков:

- 56. Не стремись использовать новейшие технологии просто потому, что это модно. Используй те инструменты, которые лучше всего подходят именно под твою задачу.

Совет про личную эффективность:

57. Заведи привычку каждый вечер подводить итоги дня и фиксировать свои достижения. Это поможет сохранять мотивацию, видеть прогресс и управлять своим ростом более осознанно.